

Postman API Testing Report

DummyJSON

Course: Software Testing

TA: Mahmoud Hadad

Team Members:

Salman Ahmed – 20221072,
Abdelrahman Mohammed – 20220195,
Khaled Mostafa - 20220120

1. GET Product Request

The screenshot shows the Postman interface with a GET request to `https://dummyjson.com/products/1`. The request is successful, returning a 200 OK status with a response time of 304 ms and a body size of 1.62 KB. The JSON response is displayed in the Body tab, showing details for a product with ID 1, including title, description, category, price, discount, rating, stock, tags, brand, sku, weight, dimensions, warranty, shipping, and availability status.

```
GET https://dummyjson.com/products/1
```

200 OK · 304 ms · 1.62 KB

```
{
  "id": 1,
  "title": "Essence Mascara Lash Princess",
  "description": "The Essence Mascara Lash Princess is a popular mascara known for its volumizing and lengthening effects. Achieve dramatic lashes with this long-lasting and cruelty-free formula.",
  "category": "beauty",
  "price": 9.99,
  "discountPercentage": 19.48,
  "rating": 2.56,
  "stock": 99,
  "tags": [
    "beauty",
    "mascara"
  ],
  "brand": "Essence",
  "sku": "BEA-ESS-ESS-001",
  "weight": 4,
  "dimensions": {
    "width": 15.14,
    "height": 13.08,
    "depth": 22.99
  },
  "warrantyInformation": "1 week warranty",
  "shippingInformation": "Ships in 3-5 business days",
  "availabilityStatus": "In Stock",
  "reviews": [
    {
      "rating": 3,

```

The screenshot shows the Postman interface with a GET request to `https://dummyjson.com/products/`. The request is successful, returning a 200 OK status with a response time of 54 ms and a body size of 7.86 KB. The JSON response is displayed in the Body tab, showing an array of product objects, each containing details like id, title, description, category, price, discount, rating, stock, tags, brand, sku, weight, dimensions, warranty, shipping, and availability status.

```
GET https://dummyjson.com/products/
```

200 OK · 54 ms · 7.86 KB

```
{
  "products": [
    {
      "id": 1,
      "title": "Essence Mascara Lash Princess",
      "description": "The Essence Mascara Lash Princess is a popular mascara known for its volumizing and lengthening effects. Achieve dramatic lashes with this long-lasting and cruelty-free formula.",
      "category": "beauty",
      "price": 9.99,
      "discountPercentage": 19.48,
      "rating": 2.56,
      "stock": 99,
      "tags": [
        "beauty",
        "mascara"
      ],
      "brand": "Essence",
      "sku": "BEA-ESS-ESS-001",
      "weight": 4,
      "dimensions": {
        "width": 15.14,
        "height": 13.08,
        "depth": 22.99
      },
      "warrantyInformation": "1 week warranty",
      "shippingInformation": "Ships in 3-5 business days",

```

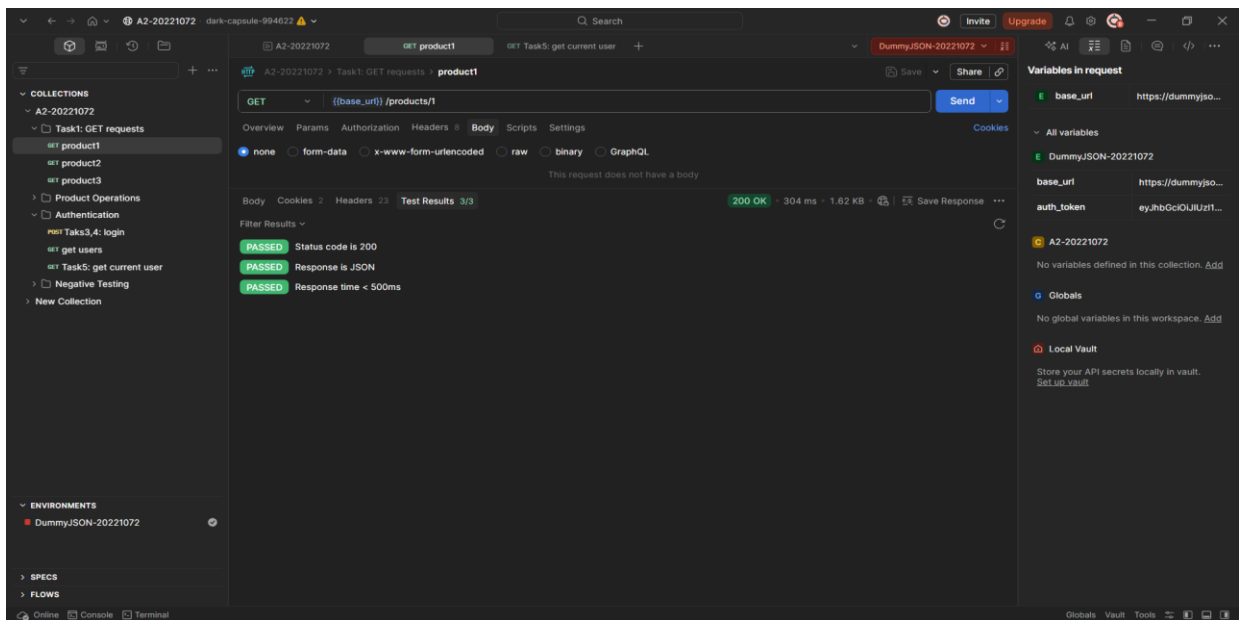
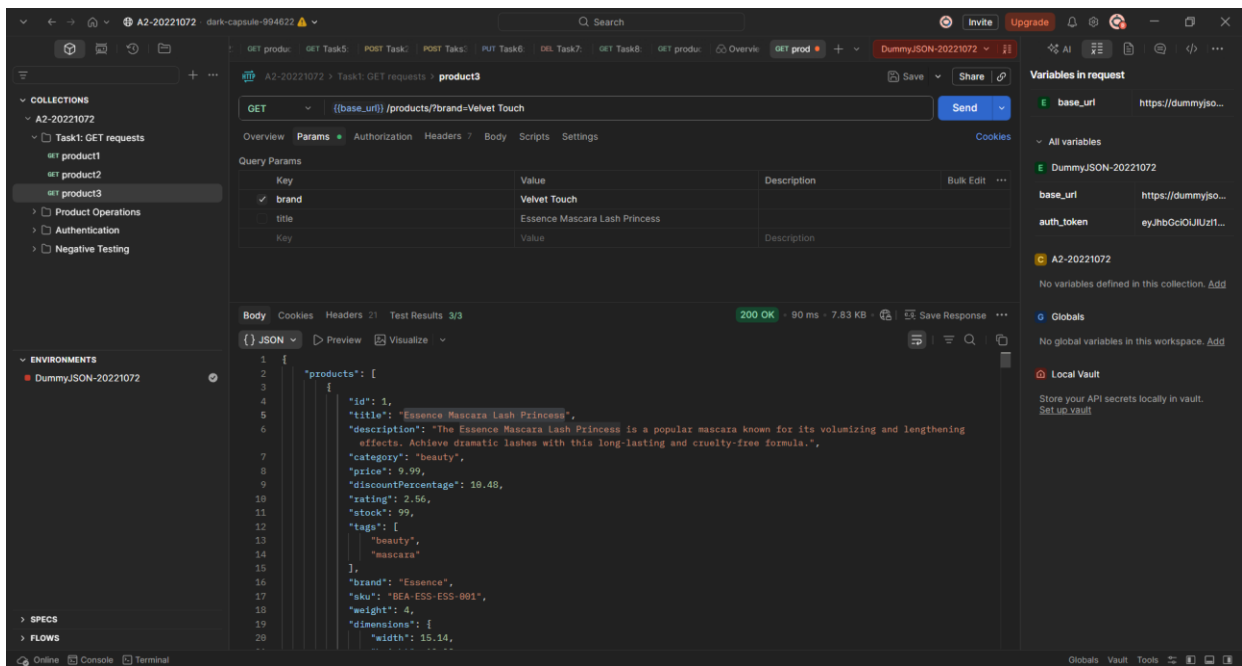


Figure 1: GET Product request with validation tests (Status 200, JSON response, response time)

2. POST Add Product

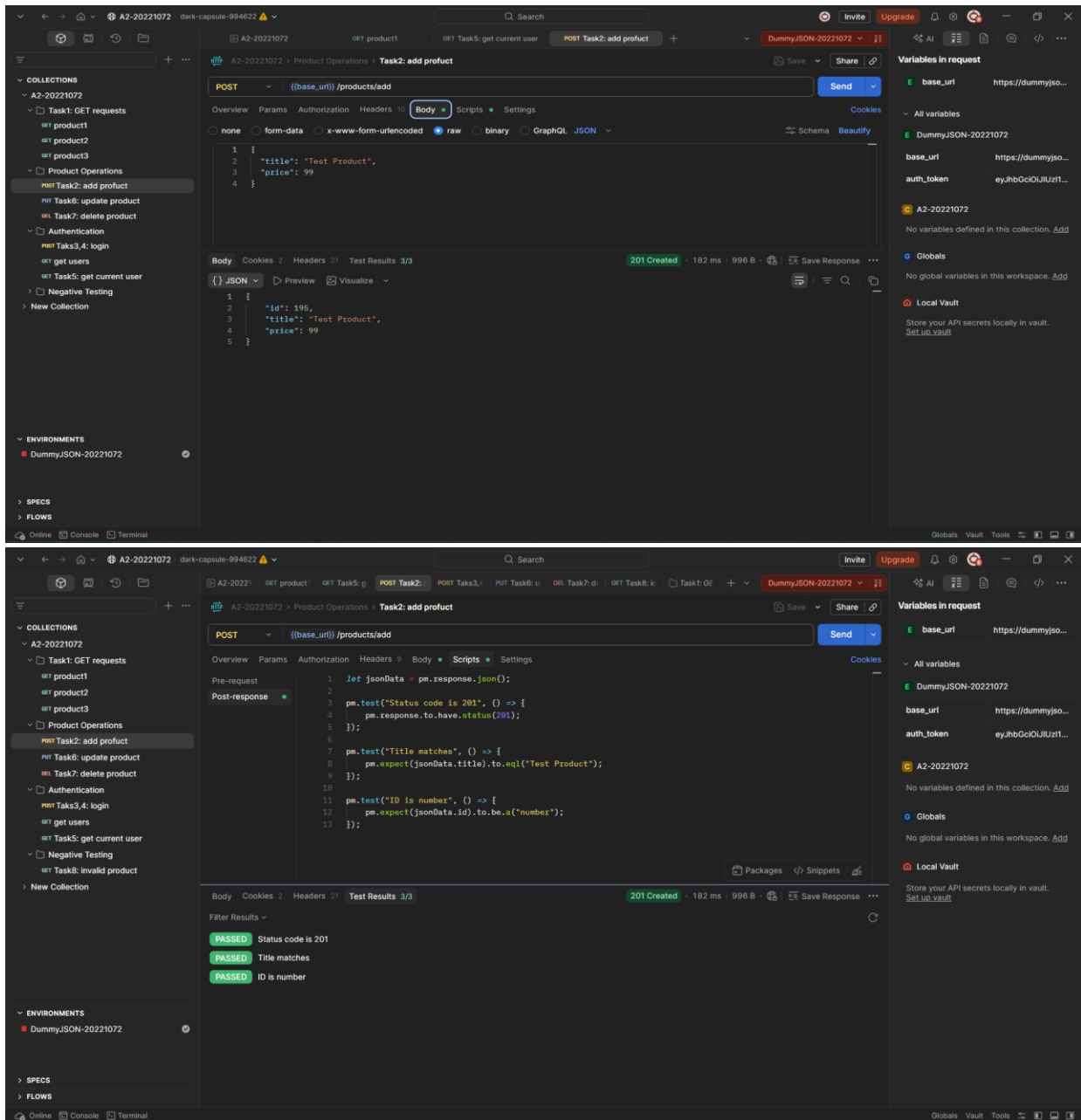
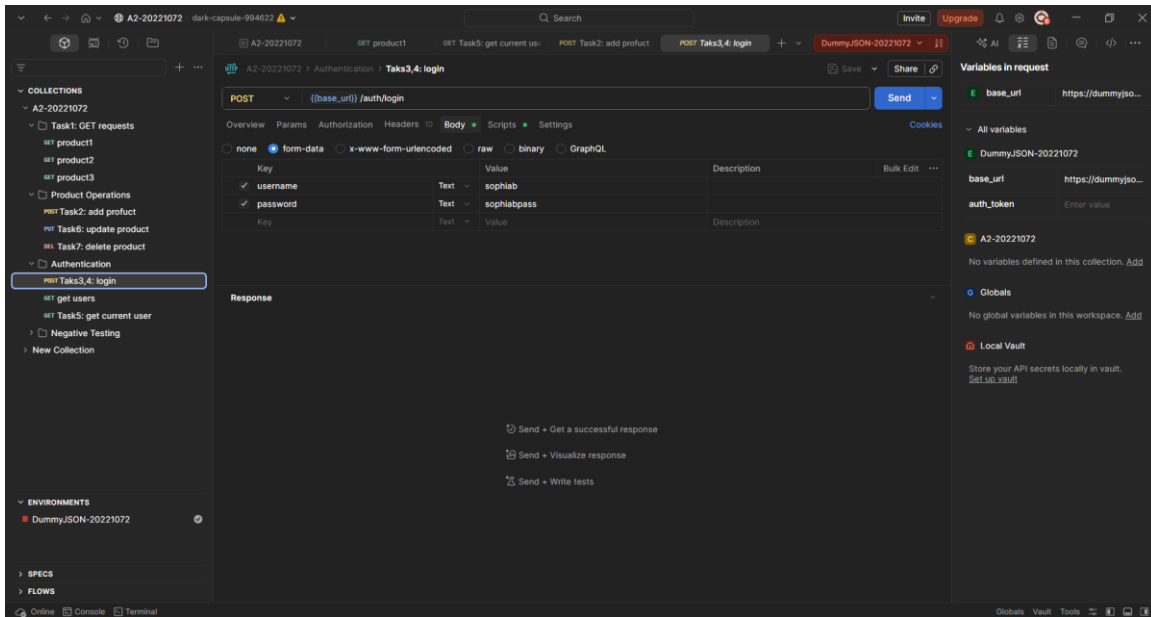


Figure 2: POST Add Product with validation of title and ID

Before Requesting



After requesting

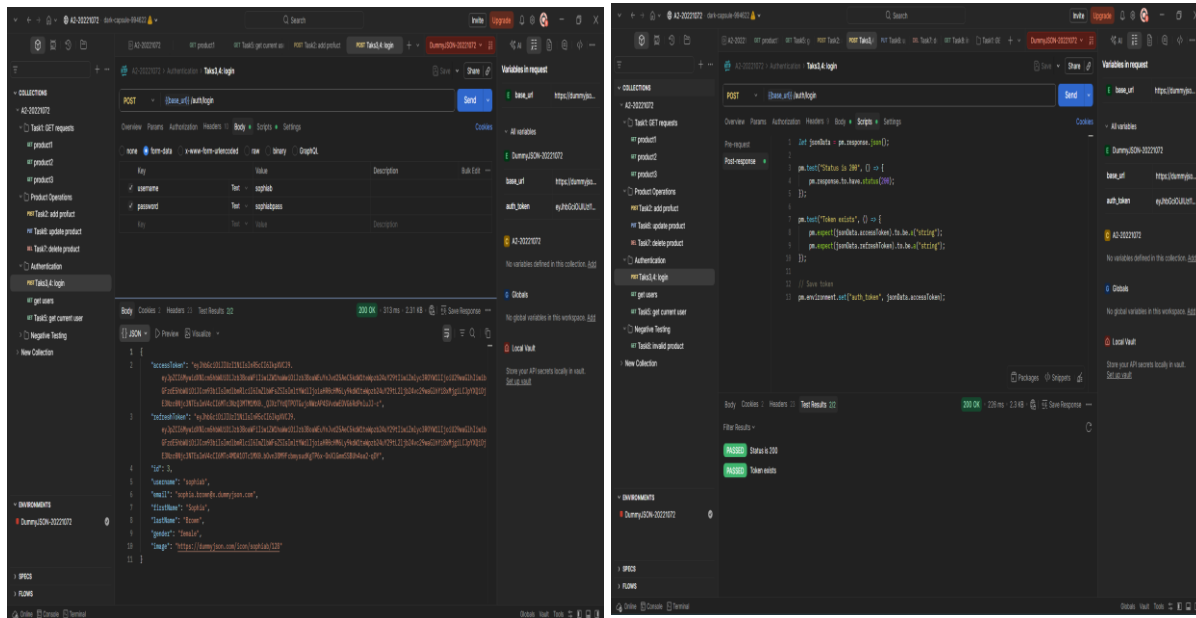


Figure 3: Login request validating token generation and saving to environment

4. Environment Variables

Before

Variables in request

E base_url	https://dummyjso...
-------------------	---------------------

▼ All variables

E DummyJSON-20221072

base_url	https://dummyjso...
auth_token	Enter value

C A2-20221072

No variables defined in this collection. [Add](#)

G Globals

No global variables in this workspace. [Add](#)

Local Vault

Store your API secrets locally in vault.
[Set up vault](#)

After

Variables in request

E base_url	https://dummyjso...
-------------------	---------------------

▼ All variables

E DummyJSON-20221072

base_url	https://dummyjso...
auth_token	eyJhbGciOiJIUzI1...

C A2-20221072

No variables defined in this collection. [Add](#)

G Globals

No global variables in this workspace. [Add](#)

Local Vault

Store your API secrets locally in vault.
[Set up vault](#)

Figure 4: Environment variables showing dynamically stored authentication token

5. GET Current User

Removed token from cookies and add it to Auth Header

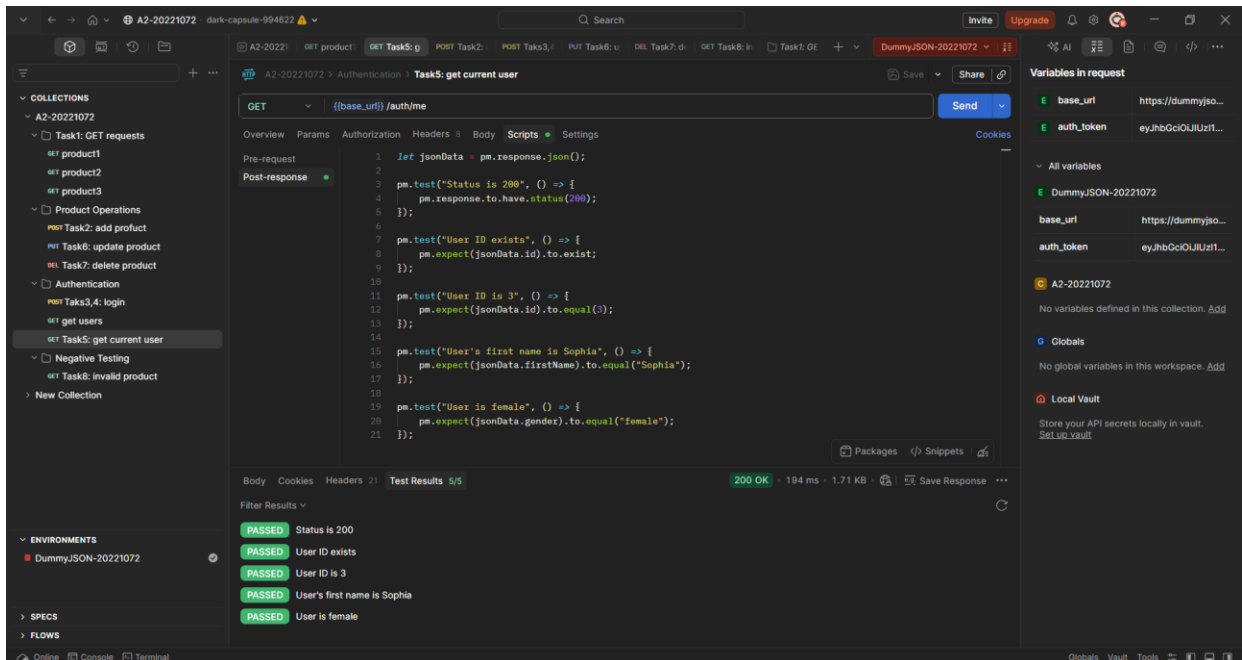
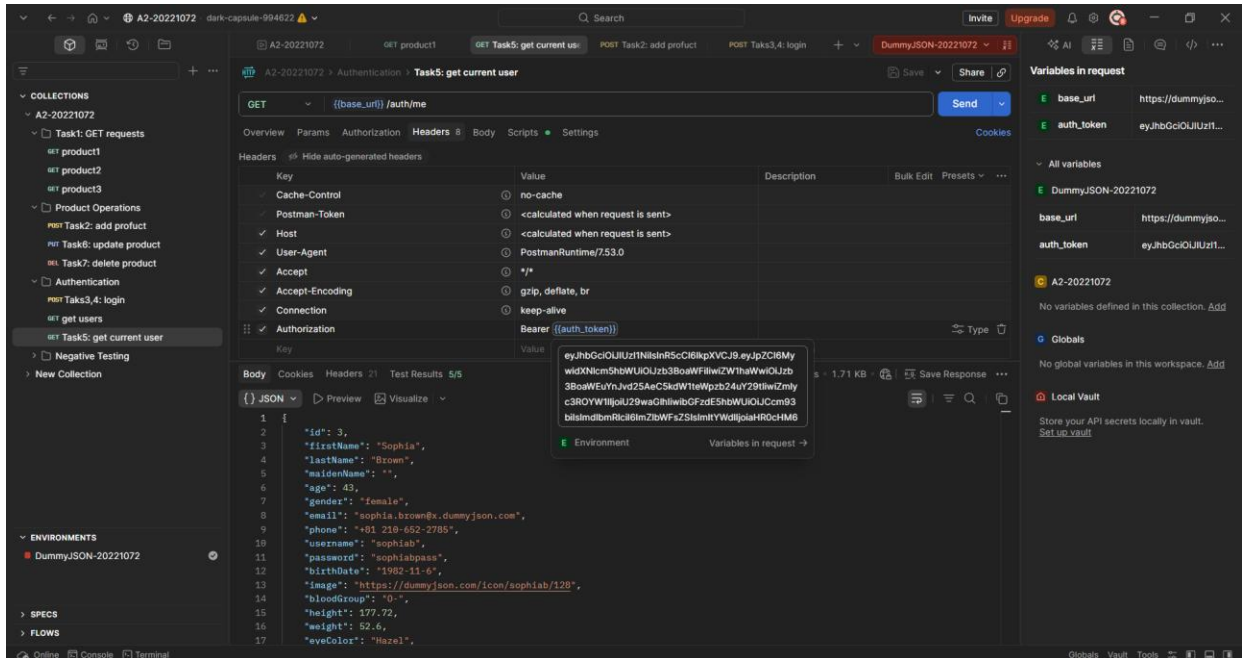


Figure 5: Authenticated request retrieving current user data

6. PUT Update Product

The image displays two screenshots of the Postman application interface, illustrating a PUT request used to update product information.

Top Screenshot: Shows the 'Task6: update product' request. The method is PUT, and the URL is `{{base_url}}/products/1`. The request body is raw JSON, containing the following data:

```
1 {
2   "title": "Updated Product",
3   "price": 99999
4 }
```

The response status is 200 OK, with a response time of 331 ms and a body size of 1.23 KB. The response body is JSON, containing the following data:

```
1 {
2   "id": 1,
3   "title": "Updated Product",
4   "price": 99999,
5   "discountPercentage": 18.48,
6   "stock": 99,
7   "rating": 2.86,
8   "images": [
9     "https://cdn.dummyjson.com/product-images/beauty/essence-mascara-lash-princess/1.webp"
10  ],
11  "thumbnail": "https://cdn.dummyjson.com/product-images/beauty/essence-mascara-lash-princess/thumbnail.webp",
12  "description": "The Essence Mascara Lash Princess is a popular mascara known for its volumizing and lengthening effects. Achieve dramatic lashes with this long-lasting and cruelty-free formula.",
13  "brand": "Essence",
14  "category": "beauty"
15 }
```

Bottom Screenshot: Shows the same 'Task6: update product' request, but with the 'Test Results' tab selected. The test results show that the status is 200, the title was updated, and the price was updated.

```
1 let jsonData = pm.response.json();
2
3 pm.test("Status is 200", () => {
4   pm.response.to.have.status(200);
5 });
6
7 pm.test("Title updated", () => {
8   pm.expect(jsonData.title).to.eql("Updated Product");
9 });
10
11 pm.test("Price updated", () => {
12   pm.expect(jsonData.price).to.eql(99999);
13 });
```

The test results show:

- PASSED Status is 200
- PASSED Title updated
- PASSED Price updated

Figure 6: PUT request updating product information

7. DELETE Product

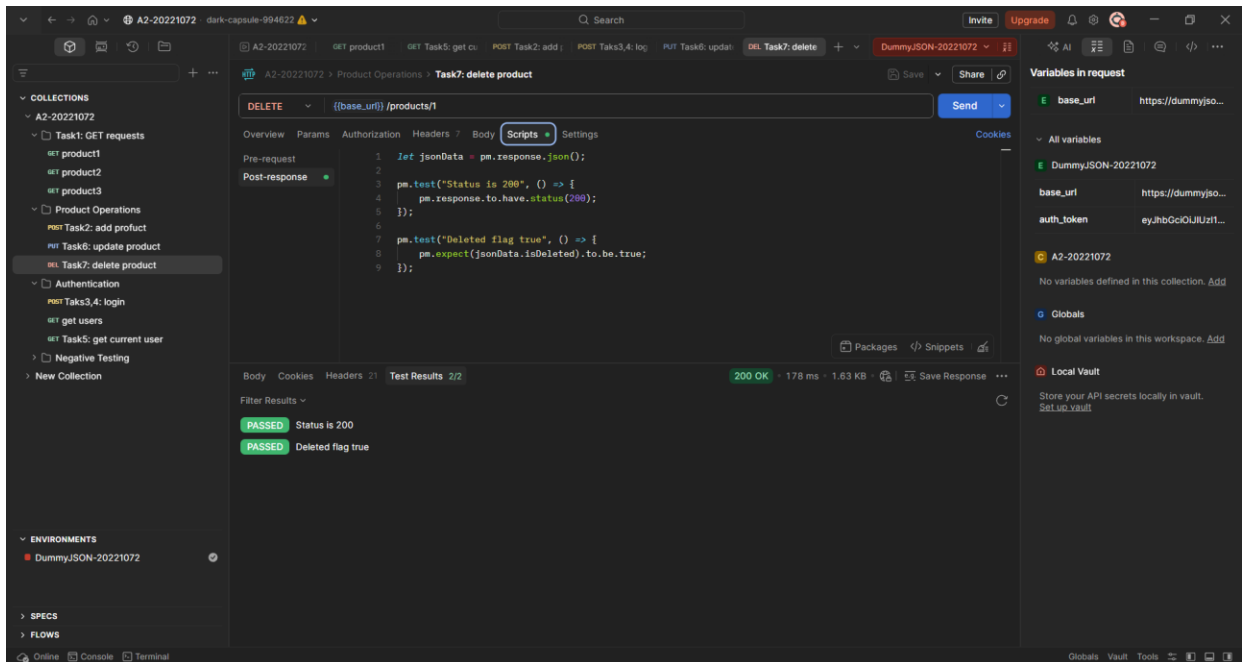
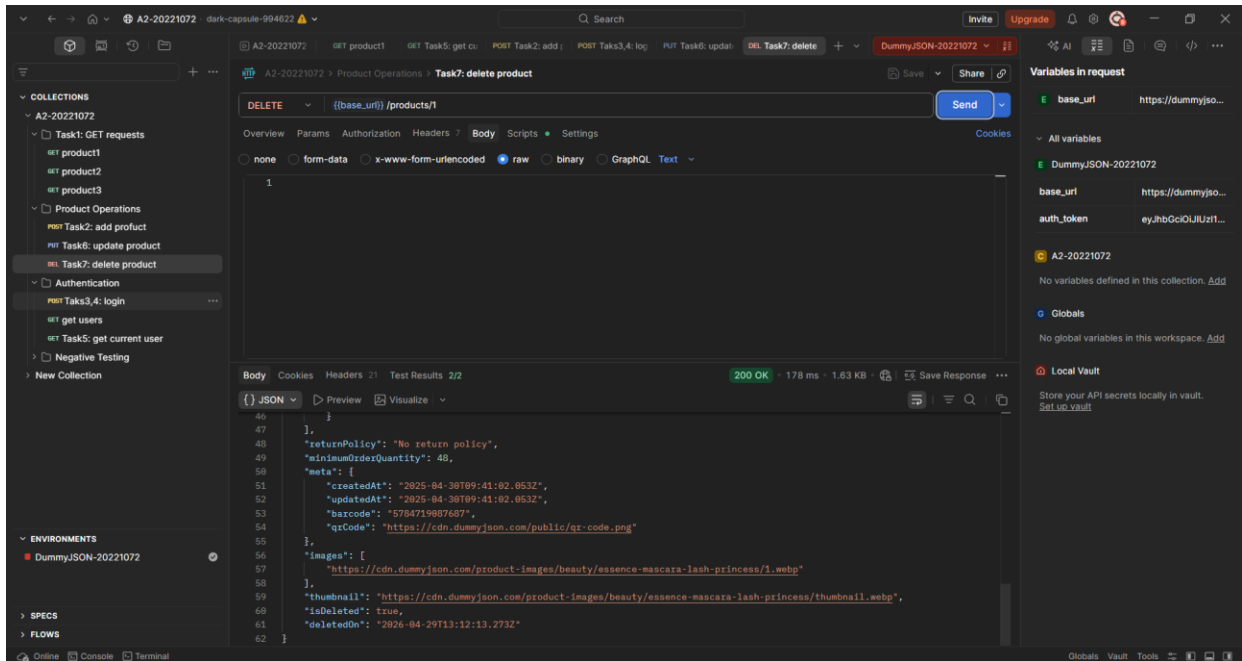


Figure 7: DELETE request confirming product removal

8. Negative Test

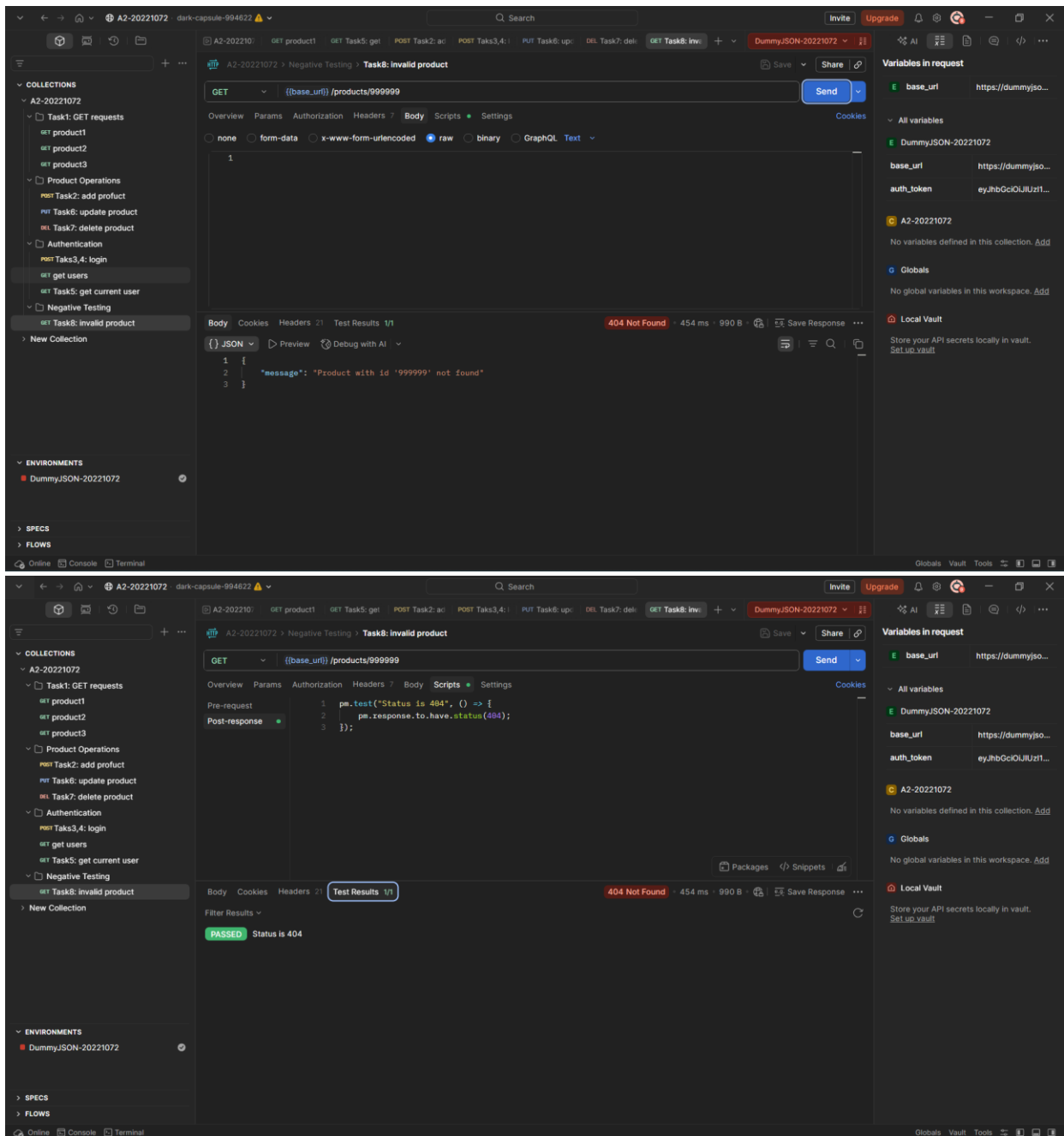


Figure 8: Negative test case validating proper error handling (404 Not Found)

9. Collection Run Results

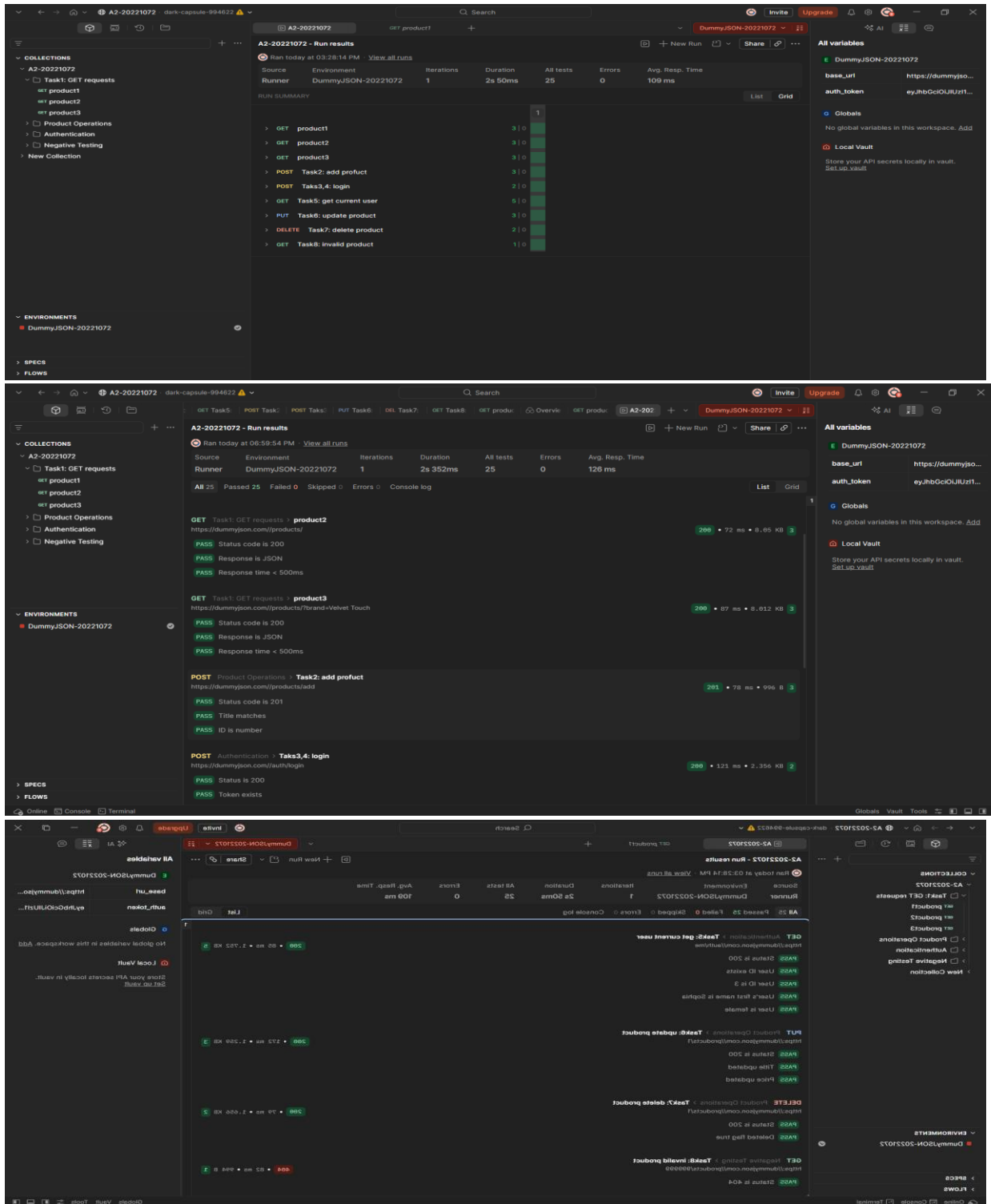


Figure 9: Full collection execution with all tests passing successfully

10. Summary

Testing Approach

This test suite was built using Postman to validate the DummyJSON API by covering CRUD operations and authentication. Automated tests were added to verify status codes, response data, and performance. Environment variables were used to handle the base URL and authentication token dynamically. Negative testing was also included to check error handling. The collection was executed using the Collection Runner to ensure all requests pass successfully in sequence.